Qdislib — Circuit Cutting Cheat Sheet

Run circuits bigger than your device: cut, execute the pieces, recombine.

Qdislib is a distributed circuit-cutting library: it splits a quantum circuit into smaller subcircuits that fit your hardware, runs them independently (in parallel, even across backends or QPUs) and recombines the results classically. It is **backend-agnostic** — the same code drives Qibo, Qiskit and CUDA-Q — and scales with PyCOMPSs.

TWO WAYS TO CUT

Gate cutting replaces a two-qubit gate (CZ / CX) by a sum of single-qubit *measure-and-prepare* operations, splitting the circuit where that gate was. **Cost: 6^k** subcircuit terms for *k* cuts.

Wire cutting cuts a qubit's wire (its timeline) into "measure here / re-prepare there". **Cost: 8^k** subcircuit terms for *k* cuts.

Every cut multiplies the work **exponentially**, so use as few, and as well-placed, as possible. Preview the blow-up with `cut_cost(cut)` before you run.

THE WORKFLOW

Get one **expectation value** <O> in a single call, or use the **three-step** form to control how each piece runs (cluster, QPU):

```
cut = qd.find_cut(circ, gate_cut=True)
val = qd.gate_cutting(circ, cut,
          observables='ZZZZ', shots=8192)

subs = qd.gate_cutting_subcircuits(
         circ, cut, observables='ZZZZ')
vals = [qd.expectation_value(s)
        for s in subs]     # run anywhere
val = subs.reconstruct(vals)
```

`estimate(circ, 'ZZZZ', max_qubits=n)` does find-cut + cut + reconstruct in one call.

QUICKSTART

```
import Qdislib as qd
from qibo import Circuit, gates

circ = Circuit(4)
circ.add(gates.H(0))
circ.add(gates.CZ(0, 1))
circ.add(gates.CZ(1, 2))
circ.add(gates.CZ(2, 3))

cut = qd.find_cut(circ, gate_cut=True,
        wire_cut=False, seed=0)
val = qd.gate_cutting(circ, cut,
        observables='ZZZZ', shots=8192)
```

CORE API

`find_cut(circ, ...)`	Choose where to cut (returns it).
`gate_cutting(circ, cut)`	<O> via gate cutting.
`wire_cutting(circ, cut)`	<O> via wire cutting.
`..._sampling(circ, cut)`	Full output distribution.
`..._subcircuits(...)`	3-step: pieces + .reconstruct().
`expectation_value(sub)`	Run one subcircuit.
`estimate(circ, ...)`	One-call cut + run + reconstruct.
`cut_cost(cut)`	Preview terms = base**#cuts.
`analytical_solution(...)`	Exact <O> (to check).
`draw_cut(circ, cut, path)`	Visualise the cut.
`from_qasm(qasm, ...)`	Import any OpenQASM 2.

SAMPLING THE DISTRIBUTION

Reconstruct the full **output distribution** `{bitstring: prob}` — to read out an image or inspect a state — instead of a single number:

```
dist = qd.gate_cutting_sampling(
         circ, cut, shots=200_000)
# three-step (parallel under PyCOMPSs):
subs = qd.gate_cutting_sampling_subcircuits(
         circ, cut)
dist = subs.reconstruct(
  [qd.sample_subcircuit(s) for s in subs])
```

`wire_cutting_sampling` too. Sampling carries a quasiprobability overhead, so it needs **more shots** than an expectation value.

FIND_CUT OPTIONS

`max_qubits=n`	Max qubits per subcircuit (device size).
`max_cuts=k`	Cap the number of cuts.
`gate_cut / wire_cut`	Which cut types to consider.
`implementation=`	'qdislib' (default) or 'ibm-ckt'.
`seed=n`	Reproducible cut selection.

OBSERVABLES

Per-qubit Pauli string over **I X Y Z** (qubit-0-first): `'ZZZZ'`, `'ZIZIZ'`, `'XZZY'`. A Hamiltonian is a weighted sum:

```
[(0.5, 'ZZII'), (-1.0, 'IIZZ')]
# or  {'ZZII': 0.5, 'IIZZ': -1.0}
```

`I` ignores a qubit — and works even on a *cut* qubit (CPU & GPU).

BACKENDS

`software=`	'qibo' | 'qiskit' | 'cudaq'.
`gpu=True`	Aer GPU (needs CUDA).
`qpu=True, qpu_dict=`	{'MN_Ona': n} / {'IBM_Quantum': 127}.
`available_backends()`	List registered backends.
`build_execution_backend`	How software/gpu/qpu pick one.
`QdBackEnd(pool)`	Load-balance across devices.
`register_backend(...)`	Plug in your own (BaseBackend).

CACHING

```
cache = qd.default_circuit_cache(
          'memory', max_entries=5000)
qd.gate_cutting(circ, cut, cache=cache)
```

The **semantic cache** reuses functionally-equivalent subcircuits (e.g. H·H·H = H). Stores: `'memory'` / `'lmdb'` / `'redis'`. `set_log_level('DEBUG')` for diagnostics.

DISTRIBUTED WITH PYCOMPSS

Qdislib is built for HPC: every subcircuit is a PyCOMPSs `@task`, so the pieces run **in parallel across cores and nodes**. The three-step workflow fans them out automatically — the same code scales from a laptop to a supercomputer.

```
import pycompss.interactive as ipycompss
ipycompss.start()   # or:  runcompss app.py
subs = qd.gate_cutting_subcircuits(circ, cut)
vals = [qd.expectation_value(s) for s in subs]
val = subs.reconstruct(vals)   # auto-parallel
ipycompss.stop()
```

GOTCHAS

- Cost is exponential in #cuts (6^k / 8^k) — fewer, well-placed cuts win; preview with `cut_cost`.
- Gate cutting cuts only **CZ / CX / CNOT** — decompose other two-qubit gates first.
- Sampling needs **more shots** than an expectation value (quasiprobability overhead).
- Identity on a cut qubit: **CPU & GPU yes**, **QPU no** (hardware returns only aggregated counts).
- Use `seed=` for a reproducible `find_cut`.